CS683 PA-1

Task 1 Report

2D Convolution

Loop Reordering · Loop Unrolling · Cache Tiling · SIMD (AVX2)

September 3, 2026

Contents

Setup **2**

1 Task 1A: Loop Reordering & Unrolling **2**
1.1 Loop Reordering 2
1.2 Loop Unrolling 2

2 Task 1B: Tiling **3**
2.1 Baseline Profiling 3
2.2 Tiled Implementation 3
2.3 Metrics and Plots 4
2.4 Questions 5

3 Task 1C: SIMD **5**
3.1 Baseline Profiling 5
3.2 Instruction Count Analysis 6
3.3 Performance 6
3.4 Width and Size Sweep 6
3.5 Questions 7

Setup

All experiments were run on an Intel 13th Gen CPU, which has both performance (P) cores and efficiency (E) cores. The P-cores have a 48 KB L1-D cache, and the E-cores have a 32 KB L1-D cache. All the measurements in this report were run on CPU 0, which is a P-core.

1 Task 1A: Loop Reordering & Unrolling

1.1 Loop Reordering

What motivated the change; what was reordered and why.

Considerations

Unit-stride access pattern; why $K \times K$ passes touch the whole image; correctness constraints.

Results

Figure 1: Speedup of `conv_reorder` over `conv_naive` vs. image size.

Figure 2: Speedup of `conv_reorder` over `conv_naive` vs. kernel size K.

Discussion

Metric used to quantify effectiveness and why; explain the "can be as slow as or slower than naive at large sizes" behavior seen at $K = 3$.

1.2 Loop Unrolling

What was unrolled (the reordered inner loop over output columns) and why (expose ILP, hide FP latency).

Considerations

Unroll factor chosen; no remainder tail needed since `W % 8 == 0`.

Results

Figure 3: Speedup of `conv_unroll` over `conv_naive` vs. image size.

Figure 4: Speedup of `conv_unroll` over `conv_naive` vs. kernel size K.

Discussion

Interpret the unrolling results in light of Section 2's memory-bound behavior.

2 Task 1B: Tiling

2.1 Baseline Profiling

The CPU we used to run is an Intel hybrid P-core/E-core design. The L1d cache size, as seen by doing `cat /sys/devices/system/cpu/cpu0/cache/index0/size`, came out to be 48 KB for the P-core, and 32 KB for the E-core. All measurements below were taken while running on a P-core (CPU 0) with `taskset`, so the L1d size for this baseline is **48 KB**.

```
taskset -c 0 perf stat -r 5 \
    -e instructions,L1-dcache-loads,L1-dcache-load-misses \
    ./bin/conv naive
```

Metric	Value (5-run mean)
Instructions	4,371,265,012
L1d loads	892,590,815
L1d load misses	571,072
Time elapsed (s)	0.222255 ± 0.002410

Table 1: `perf stat` results for `conv_naive` on a 2048×2048, $K = 3$ workload, pinned to P-core CPU 0.

$$\text{MPKI} = \frac{\text{L1-D load misses}}{\text{instructions}} \times 1000 = \frac{571{,}072}{4{,}371{,}265{,}012} \times 1000 \approx 0.131$$

Key Insight

An MPKI of ≈ 0.13 is pretty low, and it's consistent with what the naive kernel is actually doing. For every output pixel, it just needs a tiny 3×3 neighborhood of the input and the same 3×3 kernel it already used one pixel ago. All of that easily fits into the cache. So even though we loop through the entire 16 MB image over the course of the run, at any given moment it's only really touching a small, reused patch of memory. Hence, we don't get as many misses in this case.

2.2 Tiled Implementation

`conv_tile` blocks the output into square $T \times T$ tiles, running all $K \times K$ taps for a tile before moving on, so the tile stays resident in L1-D instead of being re-swept from DRAM $K \times K$ times like `conv_reorder`. We swept T on the workload (2048×2048, $K = 3$):

T	Time (ms)	Speedup	L1-D MPKI
8	18.580	0.92×	0.920
16	17.451	0.93×	21.533
24	15.642	1.03×	24.495
32	16.225	1.05×	19.624
48	15.073	1.16×	16.421
64	**14.809**	**1.10×**	**14.148**
80	15.479	1.06×	9.852

Table 2: `conv_tile` tile-size sweep, pinned to P-core CPU 0.

$T = 64$ is fastest and fits the 48 KB L1-D (≈ 33.4 KB working set), so it's what's committed in `conv_tile.cpp`.

Key Insight

Fastest ($T = 64$) isn't the same as fewest misses ($T = 8$, MPKI ≈ 0.92): tiny tiles avoid conflict misses but pay for it in loop overhead, T from 16 to 48 all have high MPKI but $T = 64$ is the best tradeoff between the two. $T = 80$ has an even lower MPKI, but its ≈ 52 KB working set no longer fits the 48 KB L1-D, so the capacity misses it introduces cost more in real latency than the MPKI ratio alone shows, making it slower than $T = 64$ despite "looking" more cache-efficient.

2.3 Metrics and Plots

We swept image size $\in \{128, 256, 512, 1024, 2048, 4096\}$ ($K = 3$) for three tile sizes – $T = 8$ (best MPKI), $T = 24$ (worst, mid conflict-miss zone), and $T = 64$ (committed) – against the naive baseline:

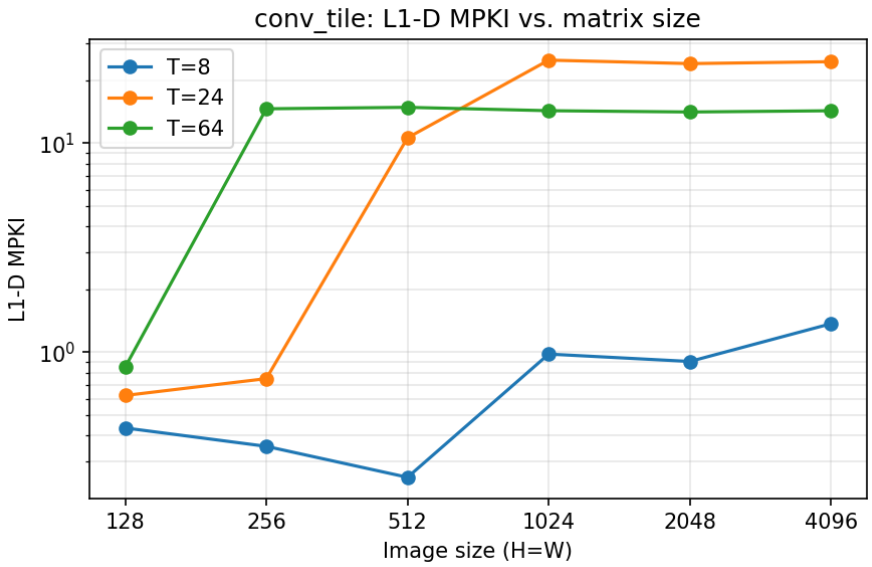

Figure 5: L1-D MPKI vs. matrix size, one line per tile size.

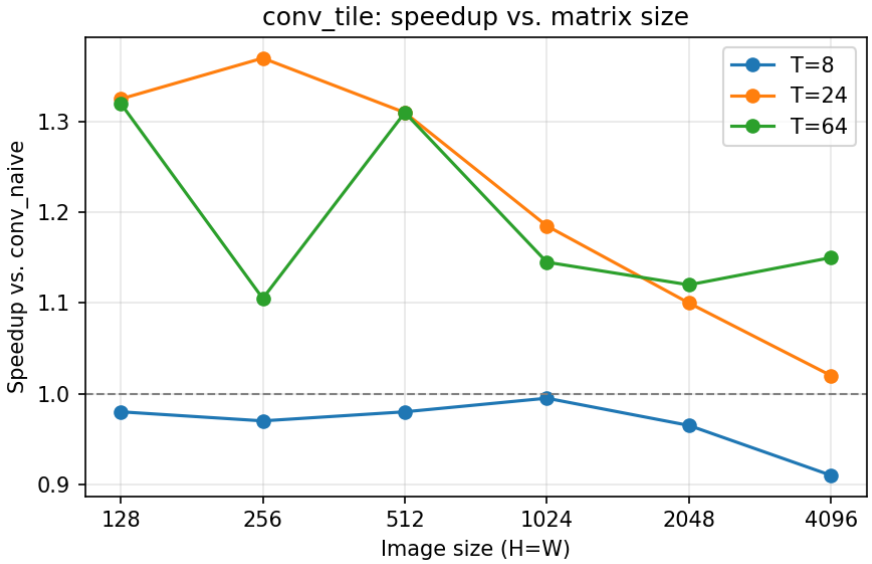

Figure 6: Speedup vs. matrix size, one line per tile size.

2.4 Questions

1. **MPKI change naive → tiled:** The MPKI, as we increase the tile size, gets worse than the naive implementation. This could be because the naive implementation is already pretty cache-optimal at $K = 3$. It benefits from long, contiguous horizontal memory accesses, which the prefetcher could easily predict and optimize. By using tiling, these contiguous accesses are broken into short burst accesses. Moreover, the working set for the naive approach probably already fits well into the L1D cache.

2. **MPKI vs. matrix size/tile size:** For $T = 8$, L1-D MPKI remains consistently low across all matrix sizes. For $T = 24$ to $T = 64$, MPKI starts low but spikes drastically as the matrix grows and finally plateaus. This is because, at the smallest matrix size, the entire working set fits into L1-D cache so misses are rare. As the matrix grows, the data exceeds L1-D capacity. The $T = 8$ tile size keeps its active working set small enough to remain in cache. But larger tile sizes introduce vertical memory jumps that exceed cache capacity, causing misses and moreover, it isn't prefetcher friendly.

3. **Speedup achieved:** Speedup was achieved only for larger tile sizes (> 24) with the sweet spot being $T = 64$. The performance improved despite high L1-D misses, indicating that tiling successfully improved temporal locality was improved which led to lower latency even if L1-D misses made it fetch data from LLC. This could be because we don't have to go to L3 or DRAM to fetch something we use in a "short" span of time because we reuse that data often (temporal locality). Also the number of instructions of tiling is about half that of the naive implementation so that is also a factor which contributes to the speedup. One of the ways to make this even more efficient could be loop unrolling (to abuse spacial locality further).

3 Task 1C: SIMD

3.1 Baseline Profiling

Since we're running the naive/simd function from the main function, there will be some extra instructions counted by `perf stat` for allocation, filling of arrays etc. So, we isolated the

per-kernel instruction count with `perf record` on the 2048×2048, $K = 3$ workload, pinned to P-core CPU 0:

```
taskset -c 0 perf record -e instructions:u -o pr.naive ./bin/conv naive 2048 2048 3
perf report -i pr.naive --stdio --sort symbol --percent-limit 0.1
```

(and likewise with `./bin/conv simd 2048 2048 3` for the SIMD kernel.) In single-stage mode the main code invokes each convolution function a fixed number of times per run: **11** for `conv_naive` (one reference call in `run_config`, plus one untimed correctness call and `kWarmup`+`kReps`= 2 + 7 timed calls in the stage loop) and **10** for the stage under test. Dividing the per-symbol totals by these counts gives the per-call figures:

Code	Total instr.	Calls	Per call
`conv_naive`	3,910,423,333	11	355,493,030
`conv_simd` (256-bit)	435,688,491	10	43,568,849

Table 3: No. of instructions for naive and SIMD optimizations

3.2 Instruction Count Analysis

The 256-bit SIMD code executes ≈ 43.6 M instructions per call against the naive code's ≈ 355.5 M (≈ 8.2× reduction), essentially the 8-wide vector width.

3.3 Performance

Stage	Time (ms)	GFLOP/s	Speedup
`conv_naive` (ref)	14.779	5.11	1.00×
`conv_simd` (256-bit)	2.327	32.44	6.35×

Table 4: `./bin/conv simd` on 2048×2048, $K = 3$, seed 1234, P-core CPU 0.

3.4 Width and Size Sweep

We swept the image size ∈ {128, 256, 512, 1024, 2048, 4096} ($K = 3$, seed 1234) for each SIMD register width, measuring the speedup of `conv_simd` over `conv_naive` at every point. The 256-bit code is the main one. The 512-bit version is skipped as none of our CPUs supported 512-bit SIMD registers :(

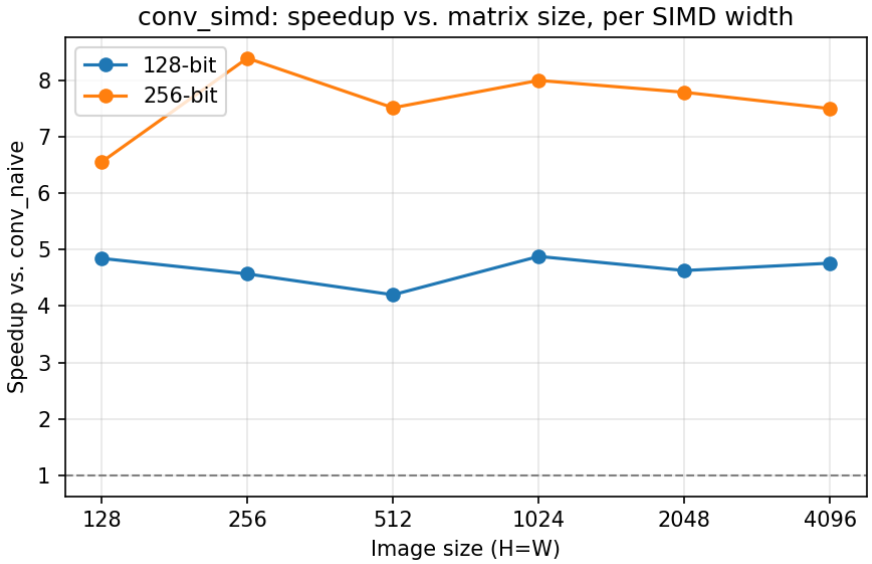

Figure 7: Speedup of `conv_simd` over `conv_naive` vs. matrix size, one line per SIMD width ($K = 3$, P-core CPU 0).

3.5 Questions

1. **Instruction count change (naive → SIMD):** report the observed change and justify it.
2. **Speedup:** how much was achieved, what contributed to it (or what limited it).
3. **Intrinsics used:** which SIMD intrinsics, and the justification for each choice.

Answer the three questions above using Tables 3, 4 and Figure 7.